

Nome: Arthur Saraiva de França 3B.

Atividade de registro semana 8

Conceitos:

Arquitetura monolítica é um modelo em que todo o sistema fica em um único projeto. Ela é mais simples no início, mas pode dificultar a manutenção e escalabilidade.

Arquitetura de microsserviços divide o sistema em vários serviços independentes. Isso melhora organização, manutenção e escalabilidade, porém aumenta a complexidade do sistema.

Design Patterns:

Singleton:

Garante que uma classe tenha apenas uma instância.

Factory Method:

Cria objetos sem depender diretamente da classe concreta.

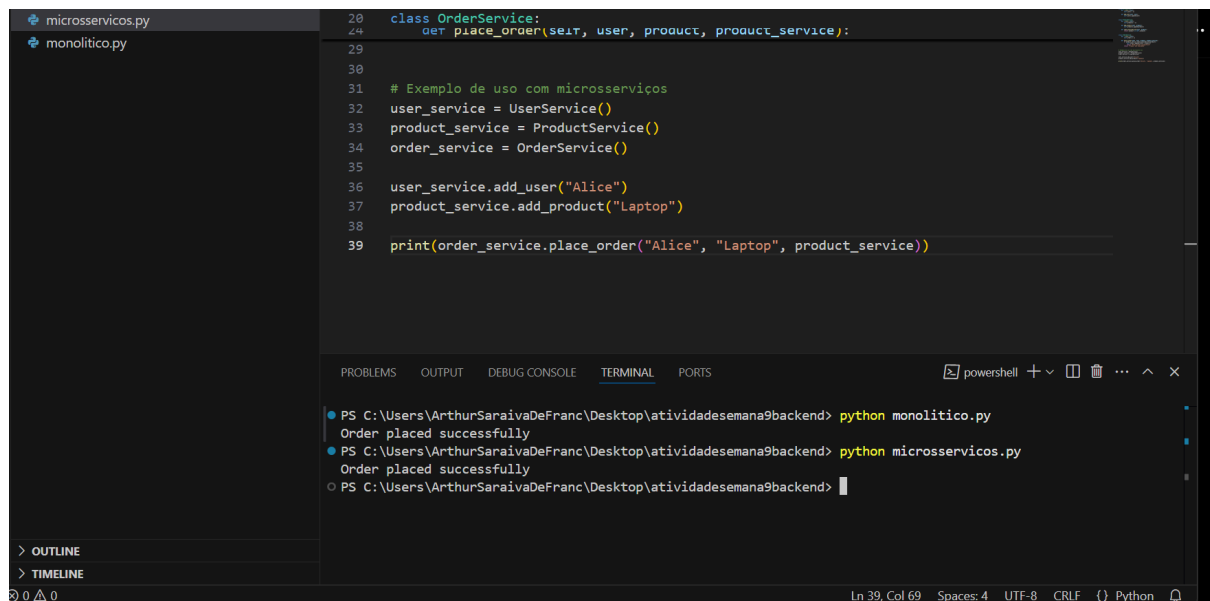
Service Registry:

Permite localizar microsserviços dentro do sistema.

Circuit Breaker:

Evita falhas em cascata entre serviços.

Com base nos códigos:



```
microservicos.py
monolitico.py

28 class OrderService:
29     def place_order(self, user, product, product_service):
30
31     # Exemplo de uso com microsserviços
32     user_service = UserService()
33     product_service = ProductService()
34     order_service = OrderService()
35
36     user_service.add_user("Alice")
37     product_service.add_product("Laptop")
38
39     print(order_service.place_order("Alice", "Laptop", product_service))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\ArthurSaraivaDeFranc\Desktop\atividadesemana9backend> python monolitico.py
Order placed successfully
PS C:\Users\ArthurSaraivaDeFranc\Desktop\atividadesemana9backend> python microservicos.py
Order placed successfully
PS C:\Users\ArthurSaraivaDeFranc\Desktop\atividadesemana9backend>

Ln 39, Col 69 Spaces: 4 UTF-8 CRLF {} Python
```

COMPARAÇÃO ENTRE ARQUITETURAS MONOLÍTICA VS MICROSERVIÇOS		
ASPECTO	ARQUITETURA MONOLÍTICA	ARQUITETURA DE MICROSERVIÇOS
 ESTRUTURA	• Todo o sistema está em um único projeto/aplicação.	• O sistema é dividido em vários serviços independentes e especializados.
 DESENVOLVIMENTO	• Mais simples no início, pois tudo está junto e integrado.	• Mais complexo no início devido à distribuição e comunicação entre serviços.
 ESCALABILIDADE	• Escalabilidade limitada, geralmente toda a aplicação precisa ser escalada.	• Alta escalabilidade, cada serviço pode ser escalado de forma independente.
 MANUTENÇÃO	• Mais difícil à medida que o sistema cresce, pois o código fica mais acoplado.	• Mais fácil de manter, pois cada serviço tem uma responsabilidade específica.
 IMPLANTAÇÃO (DEPLOY)	• Implanta-se toda a aplicação mesmo para pequenas alterações.	• Cada serviço pode ser implantado de forma independente.
 RESILIÊNCIA	• Uma falha pode comprometer toda a aplicação.	• Falhas em um serviço não impactam todo o sistema.
 TECNOLOGIAS	• Geralmente utiliza uma única tecnologia para todo o sistema.	• Permite o uso de diferentes tecnologias em cada serviço.
 CUSTO	• Menor custo inicial de infraestrutura e gerenciamento.	• Maior custo inicial devido à infraestrutura distribuída e orquestração.

Como a implementação de microserviços mudou a maneira como você pensou sobre a divisão do código?

R:Os microserviços mostraram a importância de dividir o sistema em partes independentes, deixando o código mais organizado e fácil de manter.

Quais desafios surgiram ao tentar refatorar o sistema para microserviços?

R:O principal desafio foi separar corretamente as responsabilidades dos serviços e fazer a comunicação entre eles.

Como os padrões de design ajudaram a estruturar melhor o código?

R:Os padrões ajudaram a organizar melhor o sistema, facilitando manutenção, reutilização e escalabilidade.